

Stata Basics

1. **Interacting with Stata** Submit commands via the Command window or menu bar (interactive mode), or through the do-file editor (batch mode).

- Command, Review, Variables, Properties, and Results windows
- Page up in the Command window to pull up recent commands
- Double-clicking recent commands in the Review window will bring them into Command

2. **Getting help** Basic syntax:

```
help
help keyword
help regress
help summarize
```

3. **Using Stata as a calculator** The display command evaluates expressions. Note the square root `sqrt()` is one of Stata's many functions.

```
display 2 + 2
display sqrt(49) + 10
```

Abbreviations. Stata will accept abbreviated commands. You can use as few letters as you like, so long as the abbreviation remains unique to one command. If you use abbreviations, I recommend using at least as many letters as are needed for others reading your syntax to recognize the command.

```
di 2 + 2
dis sqrt(49) + 10
```

Common mathematical operators include +, -, *, and /. The exponent symbol is ^, for example: 7^2 . For longer expressions, use parentheses to make the intended order of operations clear. For example: `di (7+2)*4`. Type `help functions` to see a complete list of functions and operations.

The Expression Builder window is useful for constructing more complicated calculations that make use of operations, functions, saved results, and more. This is found under *Data* → *Other Utilities* → *Hand Calculator* → *Create*.

You may also use `display` for text expressions, for example:

```
display "hello"
```

4. **Opening a Stata data file** To open a Stata-format (.dta) file:

```
use C:\data\myfilename.dta
use http://www.stata-press.com/data/r9/census2
```

File names with spaces must be in quotes. I generally use quotes whether they are required or not.

```
use "C:\data\my file name.dta"
```

The `clear` option tells Stata to remove any dataset already in memory and replace it with the specified one. Use this with caution, since unsaved changes will be lost. Most Stata commands have options that can be specified at the end of the line following a comma:

```
use C:\data\myfilename.dta, clear
```

Stata has some built-in datasets you can use to learn Stata. Type `help dta_examples` to see a list. You can open sample datasets directly using `sysuse`. A very common one is `auto.dta`:

```
sysuse auto.dta
```

- 5. Saving a Stata data file** If the file already exists and you want to replace it with the version currently in memory, use the `replace` option.

```
save "C:\data\my file name.dta"
save "C:\data\anotherfile.dta", replace
```

- 6. Closing a dataset** To remove a dataset from memory, use `clear`. This takes it out of memory but does not delete it from your disk. As with any other file, don't clear data from memory if you have changes that you intended to save.

```
clear
```

- 7. Reviewing the contents of a dataset** Use the Variables and Properties windows, the Variables Manager (*Data* → *Variables Manager*), and/or commands like these:

```
browse           // view the data like a spreadsheet
edit             // edit the data like a spreadsheet (not usually advised)
list             // list entire dataset in Results (not usually advised)
list in 1/10     // list observations 1-10 only
browse var1 var2 // view only var1 and var2
list var1 var2   // list only var1 and var2
describe         // basic information about variables
describe varnames // information about specific variables
desc, short      // abbreviated describe command + option
codebook varname // more information about a variable
inspect varname  // more information about a variable
count            // number of observations
count if condition // number of observations meeting a condition
```

Stata is case-sensitive. You must type variable names exactly as they appear in the Variables window.

Numeric variables are stored in different ways, depending on the size and precision of values contained in the variable. The integer types (in order of size) are byte, int, and long. The real-number types (in order of precision) are float and double. String variables are non-numeric.

8. Simple descriptive statistics, tables, and frequency distributions

```
summarize          // descriptive stats for all variables
sum varnames       // descriptive stats for select variables
sum varnames, detail // additional stats provided
tabulate varname(s) // frequency distributions
table varname(s)   // this command has many options
```

9. **if and in clauses** The if clause tells Stata to execute a command only for cases where the condition applies. The in clause tells Stata to execute the command only for cases within the specified range of observation numbers (rows). Use the latter with caution, since the sort order of your data can change. Examples:

```
summarize varname if varname2=="Y"
summarize varname if varname2!="N"
sum varname if varname2>=500
list varlist if _n < 40          // _n refers to observation number
```

Equality conditions require two equals signs. An inequality condition ("not equal to") requires ~= or !=. Conditions can include > or <, as well as greater-than-or-equal-to (>=) and less-than-or-equal-to (<=). You can also include AND or OR conditions in a statement, where AND is designated by & and OR is designated by |:

```
sum enroll if state=="NY" | state=="NJ" | state=="CT"
sum puptch if state=="NY" & enroll>250
```

Again, notice the double equals signs. Be very careful when specifying the logic of a complex if clause. Use parentheses when needed. For example, the following two statements will not yield the same result:

```
sum enroll if state=="NY" | state=="NJ" & enroll>1000
sum enroll if (state=="NY" | state=="NJ") & enroll>1000
```

Missing values Stata has special notation for missing values that depends on whether the variable is numeric or a string. If numeric, missing values are represented by a period. If string, missing values are represented by a pair of double quotes:

```
count if numvar1==.
summ var2 if numvar1!=.
list var1 var2 if strvar1=="
```

10. **Designating the order of variables** This is useful when your dataset contains a large number of variables. You can also sort the viewing order of variables temporarily in the Variables window by clicking on the column you wish to sort by (e.g., variable name or variable label).

```
order varlist
aorder          // alphabetizes list of variables
```

The order of variables in the Variables window is important when you wish to use variable lists as shorthand. For example, if variables var1 through var5 are sorted in alphabetical order, then the

following two commands are equivalent:

```
sum var1-var5
sum var1 var2 var3 var4 var5
```

Stata refers to `var1-var5` as a “varlist.” Another type of varlist uses the wildcard symbol:

```
sum var*           // all variables starting with "var"
```

11. Dropping or keeping variables and cases

```
drop varlist        // drop specified variables
drop if condition   // drop cases meeting a condition
keep varlist        // keep only specified variables
keep if condition   // keep only cases meeting a condition
```

Dropping variables is like deleting columns in a spreadsheet. Dropping observations/cases is like deleting rows in a spreadsheet.

12. Do-files Do-files are programs—sets of instructions that Stata will carry out when you run them. These are created in the Do-File Editor. Some useful commands often found in do-files are:

```
cd C:\mydirectory\      // set working directory
cd "C:\Data\Class"

* Comments in do file    // single-line comment
// Comments in do file  // single-line comment
/* Comments */           // multiple-line comment

version                  // display current Stata version
version 10               // require Stata version 10+
clear all                 // remove everything from memory
set more off             // prevent Stata from pausing display
set linesize 80          // set display width

# delimiter ;            // use semicolon as line delimiter
///                      // continue command on next line
# delimiter cr           // reset delimiter to hard return
```

It is possible to copy and paste commands from the Review (or Command) window into the do-file. This allows you to test your command to see whether it does what you want before including it. Do-files can be executed all at once or in subsections. To execute in subsections, highlight the section you want to run and click the Execute (do) button.

13. Mac versus PC The most notable difference between Stata on the Mac and PC is how working directories are set. PC uses backslashes, while Mac uses forward slashes:

```
cd "C:\Users\Sean"      // PC
cd "/Users/Sean"        // Mac
```

On a PC, the command `cd Sean` will return an error message. On a Mac, `cd Sean` will set the working

directory to the subfolder titled “Sean” in your current working directory, assuming it exists.

The command `pwd` displays the current working directory on both PC and Mac. On the PC, `cd` without arguments also does this. On the Mac, however, `cd` without arguments when you are in a subfolder changes the working directory one level up.

The following code in a do-file will help you move between Mac and PC:

```
if regexm(c(os),"Mac")==1 {
    local mypath = "/Users/Sean"
}
else if regexm(c(os),"Windows")==1 local mypath = "\Users\Sean"
cd "`mypath`"
```

- 14. Creating and viewing log files** Logs can be saved as text files or in Stata markup language (`.smcl`) format.

```
log using C:\data\yourfilename.txt, text    // start text log
log using C:\data\yourfilename.smcl, smcl   // start SMCL log
log close                                  // stop logging
log off                                    // temporarily pause logging
log on                                     // resume logging
view C:\data\yourfilename.txt              // view text log
capture log close                          // ensure no log is open
cmdlog using filename                      // log commands only
cmdlog close                               // stop command log
```

The command `capture` before another Stata command tells Stata to ignore any error message that might result. For example, if there were no log file open `log close` would produce an error message.

- 15. Creating new variables** Expressions can include arithmetic operations, mathematical functions, statistical functions, random-number functions, string functions, date functions, and many other things. Type `help functions` for a complete list, or use the Expression Builder mentioned earlier.

```
generate newvar = expression
gen aplusb = vara + varb
```

When creating complicated expressions, be sure to pay attention to order of operations, as mentioned earlier. Also, pay attention to how missing values of variables used in the expression are treated.

- 16. Replacing values of existing variables** Use `replace` to change values of existing variables. The first command below divides all existing values of `var1` by 100. The second replaces the value of `state` with "NY" whenever `state` equals "New York". Note the double equals sign in the `if` condition.

```
replace var1 = var1/100
replace state = "NY" if state=="New York"
```

- 17. Labeling variables** This can also be done in the Variables Manager or Properties window.

```
label var varname "Descriptive label here"
```

18. Renaming variables and attaching notes

```
rename oldvarname newvarname
```

Some researchers prefer not to rename variables, which could risk losing information about the original variable name. Instead, a copy of the old variable can be made. It is also possible to attach notes to variables:

```
notes var1: Insert note here    // specify a note for var1
notes var1                     // view the note for var1
```

19. Numeric missing values

Stata treats numeric missing values (.) as larger than any valid numeric value, so use extreme caution when using the > operator when there are missing values. For example, the following would code elderly as 1 in cases where age is missing:

```
gen elderly = 0 if age<65
replace elderly = 1 if age>=65
```

20. Sorting data

```
sort varlist                // sort ascending by varlist
gsort -varname              // sort descending by varname
```

21. Using by groups

```
sort var1
by var1: sum var2
```

In the second statement, `sum var2` is executed separately for each unique value of `var1`. Therefore, this approach should only be used when there are a manageable number of values/categories for `var1`.

22. User-written commands

User-written commands are stored in .ado files. "SSC" is Statistical Software Components at Boston College and is a common repository for these files.

```
help nmissing
ssc install nmissing    // install the ado-file "nmissing"
```

23. The wildcard symbol

The wildcard symbol (*) allows you to apply a command to numerous variables with the same prefix or suffix. For example:

```
summarize gdp19*
drop pop*
```

Use the wildcard with caution to avoid doing things to variables you did not intend to change.

24. Debugging do-files

Stata provides error numbers that can be looked up. With a little experience, it becomes easier to catch errors. The most common bugs are errors in syntax:

- Not using correct symbols in a condition statement (e.g., double equals sign)
- Misspelling command names and/or variable names
- Capitalizing letters in variable names that should not be capitalized, or failing to capitalize letters that should be
- Spaces where none should be
- Incorrect options
- Options in the wrong place
- Variables of the wrong type for the desired command (e.g., string vs. numeric)

Some strategies for debugging:

- Try running the program in steps (sections of the do-file)
- Try running the program on different data
- Use tracing: `set trace on`
- Restart and/or update Stata: `update all`